

Module 1: Webhook Ingestion & n8n Setup

n8n Deployment & Network Setup

I built an Ubuntu Server VM named `soar-n8n` at `10.1.1.60` in the Server Zone (`vmbr1`) to host n8n. Since DHCP is turned off in this zone, I manually configured a static IP (`10.1.1.60/24`) and gateway (`10.1.1.1`).

I installed `qemu-guest-agent` and Docker, then launched n8n inside a container with persistent storage:

```
docker run -d \  
  --name n8n \  
  --restart always \  
  -p 5678:5678 \  
  -v n8n_data:/home/node/.n8n \  
  docker.n8n.io/n8nio/n8n
```

Because n8n and my Splunk server (`10.1.1.10`) sit in the same subnet, pfSense allowed them to communicate directly without extra firewall rules.

Webhook Configuration & Splunk Integration

I logged into n8n (`http://10.1.1.60:5678`) over my WireGuard VPN and created a workflow called `Project4-SOAR-Pipeline` . I added a **Webhook** trigger node set to listen for `POST` requests at `/webhook-test/splunk-alert` .

In Splunk (`10.1.1.10:8000`), I converted my path traversal search into a scheduled alert:

```
index=linux sourcetype="docker:json" "etc/passwd"
```

I scheduled the alert to run every minute (`* * * * *`), set it to trigger when results were greater than `0` , and added a **Webhook Alert Action** pointing to `[http://10.1.1.60:5678/webhook-test/splunk-alert]` (`http://10.1.1.60:5678/webhook-test/splunk-alert`) .

Settings

Alert

Suspicious Page Parameter - LFI Audit

Description

Optional

Search

index=* "page="
| table _time, host, log, stream

Alert type

Scheduled

Real-time

Run on Cron Schedule ▼

Time Range

Last 1 minute ▶

Cron Expression

e.g. 00 18 *** (every day at 6PM). [Learn More](#)

Expires

24

hour(s) ▼

Trigger Conditions

Trigger alert when

Number of Results ▼

is greater than ▼

0

Trigger

Once

For each result

Throttle ? ☐

Trigger Actions

+ Add Actions ▼

When triggered

Add to Triggered Alerts
Remove

Webhook
Remove

URL

http://10.11.60:5678/webhook/test/enlink-alert

Specified URL to send JSON payload via HTTP POST (ex., https://your.server.com/api/v1/webhook).

[Learn More](#)

Pipeline Verification

To test the pipeline, I launched a path traversal request from Kali (10.1.4.100) against the web server (10.1.2.10):

```
curl -s "http://10.1.2.10/vulnerabilities/fi/?page=../../../../../../etc/passwd" > /dev/null
```

Splunk detected the attack, fired the alert action, and delivered the raw JSON payload to n8n in under 45 seconds.

Parameters

Settings

Listen for test event

Webhook URLs

Test URL

Production URL

POST

http://localhost:5678/webhook/splunk-alert

HTTP Method

POST

Path

splunk-alert

Authentication

None

Respond

Immediately

OUTPUT

1 item

headers	params	query	body
accept-encoding : identity content-length : 1668 host : 10.11.60:5678 content-type : application/json user-agent : Splunk/88B833B8-B925-4612-A4CF-2FB535688AAA connection : close	{empty object}	{empty object}	sid : scheduler_ad7715_at_1788330 search_name : SO app : search owner : admin results_link : http://siem:8000/app/sid=scheduler_37715_at_178833 result _bkt : linux~10~2FB535688A _cd : 10:11834 _eventtype_col _indextime : 178 _raw : {"log": "10

Module 2: Sandboxed Threat Analysis & Pipeline Orchestration

Data Ingestion & SIEM Setup

I created a broad Splunk alert search (`index=* "page="`) to catch web requests using the `page=` parameter. This flags two types of events:

- **Malicious Attacks:** Path traversal attempts like `page=../../../../etc/passwd` that require AI triage and active blocking.
- **False Positives:** Safe requests like `page=include.php` that should be logged and ignored.

When Splunk fires a webhook, n8n receives the alert payload. A JavaScript node (`Code in JavaScript`) parses the source IP, URI, and raw log, then writes them directly to a shared workspace file (`/app/input.json`) using `fs.writeFileSync`. I read that using file storage avoids command-line escaping errors when passing complex log strings to Docker.

The screenshot shows an n8n workflow editor. On the left, the 'JavaScript' node is configured with the following code:

```
6  const srcIp = $json.body.result.src_ip
7  || rawLog.match(/(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})/)?.[1]
8  || "Unknown";
9
10 const uri = rawLog.match(/"GET\s+([^\s]+)/)?.[1] || "Unknown";
11
12 const payload = {
13   raw_log: rawLog,
14   extracted_ip: srcIp,
15   extracted_uri: uri,
16   timestamp: new Date().toISOString()
17 };
18
19 fs.writeFileSync('/app/input.json', JSON.stringify(payload, null,
20 2));
21 return { json: payload };
```

On the right, the 'OUTPUT' section shows the resulting JSON payload:

```
1 item
T raw_log 10.1.4.100 - - [03/Sep/2026:06:15:57 +0000] "GET
/vulnerabilities/fi/?page=../../../../etc/passwd HTTP/1.1" 302
485 "-" "python-requests/2.34.2"
T extracted_ip 10.1.4.100
T extracted_uri /vulnerabilities/fi/?page=../../../../etc/passwd
T timestamp 2026-09-03T14:48:39.007Z
```

Sandboxed Container Execution

An `Execute Command` node spins up a temporary Python container to analyze the input file safely:

```
/app/bin/docker run --rm -v /home/tonies/project4-soar-pipeline:/app python:3.11-slim
python3 /app/sandbox_analyzer.py /app/input.json
```

The script evaluates the request and outputs JSON to `stdout`:

- **Malicious Payload:** Returns `risk_score: 100` and `is_malicious: true` (flagging directory traversal sequences and sensitive file targets).
- **Benign Payload:** Returns `risk_score: 0` and `is_malicious: false`.

```
# exitCode 0
T stderr
T stdout {"risk_score": 100, "threat_level": "HIGH", "findings": ["Directory traversal sequence detected ('..')", "Sensitive file access targeted (/etc/passwd)", "Automated attack tool User-Agent identified"], "is_malicious": true}
```

Output Parsing & Threat Decision Gate

A `Parse Sandbox JSON` node converts the raw container output into native `n8n` variables. An `If` node then splits the execution path:

- **True Branch:** Sends verified attacks (`is_malicious == true`) to Module 3 for Claude AI incident reporting.
- **False Branch:** Sends safe requests (`is_malicious == false`) to a `Log Benign Event` node. This logs the audit trail and stops the workflow, saving LLM token costs and preventing false firewall blocks.

Conditions

```
fx {{ $json.is_malicious }}
is true
```

Troubleshooting & Security Setup

- **Command Line Fix:** Wrote payloads to `/app/input.json` instead of passing raw logs directly into `docker run`, fixing parsing hangs caused by special characters.
- **Binary Path Fix:** Resolved a `docker: not found` error by specifying the full binary path (`/app/bin/docker`).
- **Boolean Evaluation:** Changed the `If` node condition type from **String** to **Boolean** so it recognized Python's native `true/false` values.
- **Container Hardening:** Configured `n8n` to run under the standard non-root `node` user (UID 1000) and granted Docker socket access using host group mapping.
- **Credential Safety:** Saved the Claude API key using `n8n`'s encrypted **Header Auth** manager (`x-api-key`) instead of referencing raw `.env` files inside workflow nodes.

Module 3: AI-Assisted Threat Triage & Incident Analysis

LLM Integration & Prompt Design

To automate deep threat analysis on malicious payloads (`is_malicious == true`), I connected `n8n` to Anthropic's Messages API (`POST [https://api.anthropic.com/v1/messages]` (`https://api.anthropic.com/v1/messages`)) using an `HTTP Request` node.

I authenticated the endpoint via `n8n`'s encrypted `Header Auth` manager using the `x-api-key` header alongside the required `anthropic-version: 2023-06-01` header. I selected `claude-sonnet-4-5` as the model engine due to its rapid execution speed and high accuracy for structured log parsing.

To guarantee consistent output parsing for downstream nodes, I passed a strict system prompt instructing Claude to function as a Tier 3 SOC Analyst and output only raw JSON without markdown formatting:

```
{
  "model": "claude-sonnet-4-5",
```

```

"max_tokens": 500,
"system": "You are a Tier 3 SOC Analyst. Analyze the following sandboxed alert data.
Output your analysis strictly as a JSON object with no markdown formatting or extra text.
Use these exact keys: 'severity', 'mitre_technique', 'incident_summary', and
'remediation_action'.",
"messages": [
  {
    "role": "user",
    "content": "Analyze this alert data: " + JSON.stringify($json)
  }
]
}

```

Structured Output Extraction

Because LLMs wrap structured text in markdown blocks (``json) inside the API response array (content[0].text), I added a Parse Claude Report JavaScript node to sanitize the string:

JavaScript

```

const rawText = $json.content[0].text || "{}";
const cleanedText = rawText.replace(/```json\n?|\n?```/g, '').trim();
const parsed = JSON.parse(cleanedText);

return {
  json: {
    ...parsed,
    attacker_ip: $('Parse Sandbox JSON').first().json.extracted_ip,
    target_uri: $('Parse Sandbox JSON').first().json.extracted_uri,
    timestamp: new Date().toISOString()
  }
};

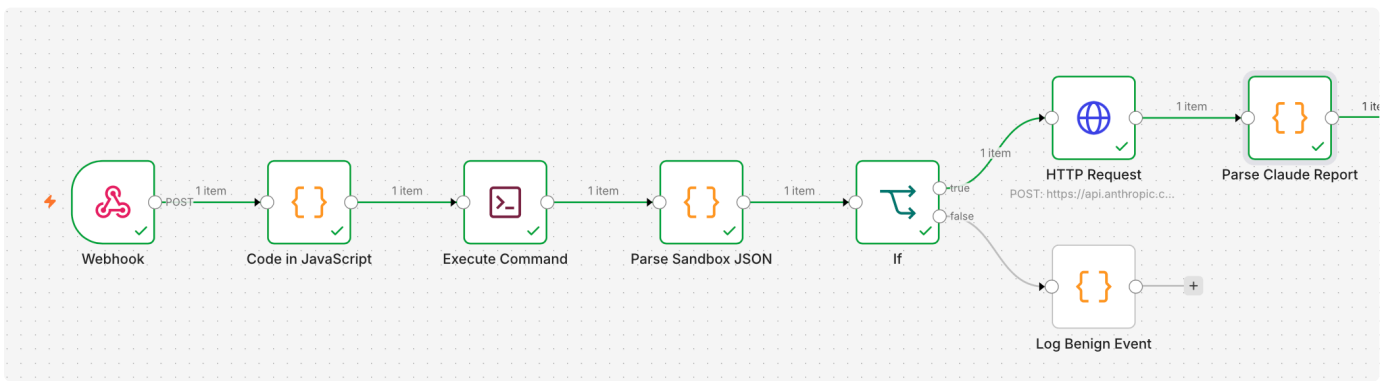
```

This node strips markdown artifacts, parses the core JSON object, and appends upstream pipeline metadata (attacker_ip , target_uri , and ISO timestamp) into a single output object.

Triage Results & MITRE ATT&CK Mapping

When evaluated against the path-traversal payload targeting /etc/passwd , Claude successfully triaged the event and populated the node output:

- **Severity:** CRITICAL
- **MITRE ATT&CK Mapping:** T1083 – File and Directory Discovery
- **Incident Summary:** Confirmed directory traversal attack attempting unauthorized access to sensitive system authentication files (/etc/passwd) using automated tooling (python-requests).
- **Remediation Action:** Recommended immediate IP isolation at the firewall level, input validation on the page parameter, and web server log audits for successful HTTP 200 exploitation responses.



End-to-end n8n SOAR pipeline execution showing verified threat triage on the True branch and automated alert suppression on the False branch.

Troubleshooting & Key Takeaways

- **Model Identifier 404s:** Initial API calls failed with a 404 resource not found error when referencing dynamic aliases (`claude-3-5-sonnet-latest`). Updating the endpoint to explicit, active model identifiers (`claude-sonnet-4-5`) resolved API routing.
- **JSON Sanitization:** Raw LLM outputs contain markdown formatting that breaks native n8n variables. Programmatically stripping code block fences using regex in JavaScript ensured reliable data flow into downstream automation nodes.

Module 4: Automated Remediation via pfSense REST API

Design Decision: Alias Updates, Not Dynamic Rules

Rather than having n8n create and destroy firewall rules on every alert, which is fragile and risks rule-ordering bugs, I designed this stage around a single static block rule that references a firewall alias. n8n only ever patches the alias's contents, so the actual enforcement logic lives in one place I can audit in the pfSense GUI at any time.

pfSense REST API Setup

pfSense has no official REST API in CE, so I installed the community-maintained **pfSense-pkg-RESTAPI** package (recently renamed from its original maintainer's repo to `pfrest/pfSense-pkg-RESTAPI`). It's no longer available through the GUI Package Manager, so I installed it via shell after enabling SSH access (disabled by default) at System > Advanced > Admin Access:

```
pkg-static add https://github.com/pfrest/pfSense-pkg-RESTAPI/releases/latest/download/pfSense-2.8.1-pkg-RESTAPI.pkg
```

Under System > REST API > Settings, I enabled the API, restricted **Allowed Interfaces** to LAN only (matched against the interface holding 10.1.1.1, confirmed via Status > Interfaces rather than the Assignments page, which only shows MAC addresses), and set Authentication Methods to **Key**. I generated a key under System > REST API > Keys. This package's auth model uses a plain header, not Bearer or Basic:

```
X-API-Key: <key>
```

No pfSense username is tied to the request at all, which took some troubleshooting to confirm since there's nowhere in the User Manager showing the key attached to an account.

I created a host alias named `SOAR_Blocklist`, seeded with a placeholder IP (pfSense won't save an alias with an empty address list), and added a block rule on the **Kali-facing interface**, not the DVWA-facing one. pfSense filters inbound at the interface where traffic enters, so blocking at the attacker's ingress point stops traffic before it's routed anywhere, and protects every internal zone at once rather than just the one target host. In a real deployment this would be the WAN interface. In the lab, Kali's interface plays that role.



API Quirks I Ran Into

The API distinguishes **plural vs. singular endpoints** in a way that isn't obvious from the docs. `GET /api/v2/firewall/aliases` (plural, no params) lists everything, while single-item operations use `GET/PATCH /api/v2/firewall/alias?id=N`, and critically, that `id` is the alias's *numeric* index, not its name. Passing `name=SOAR_Blocklist` instead of `id=1` fails with `MODEL_REQUIRES_ID` or `MODEL_OBJECT_NOT_FOUND`.

I also hit a 406 error from n8n's HTTP Request node until I explicitly added an `Accept: application/json` header. The API rejects n8n's default multi-type Accept header outright. PATCH requests also need `"apply": true` in the body, or the alias change sits in pfSense's pending-changes queue indefinitely instead of taking effect.

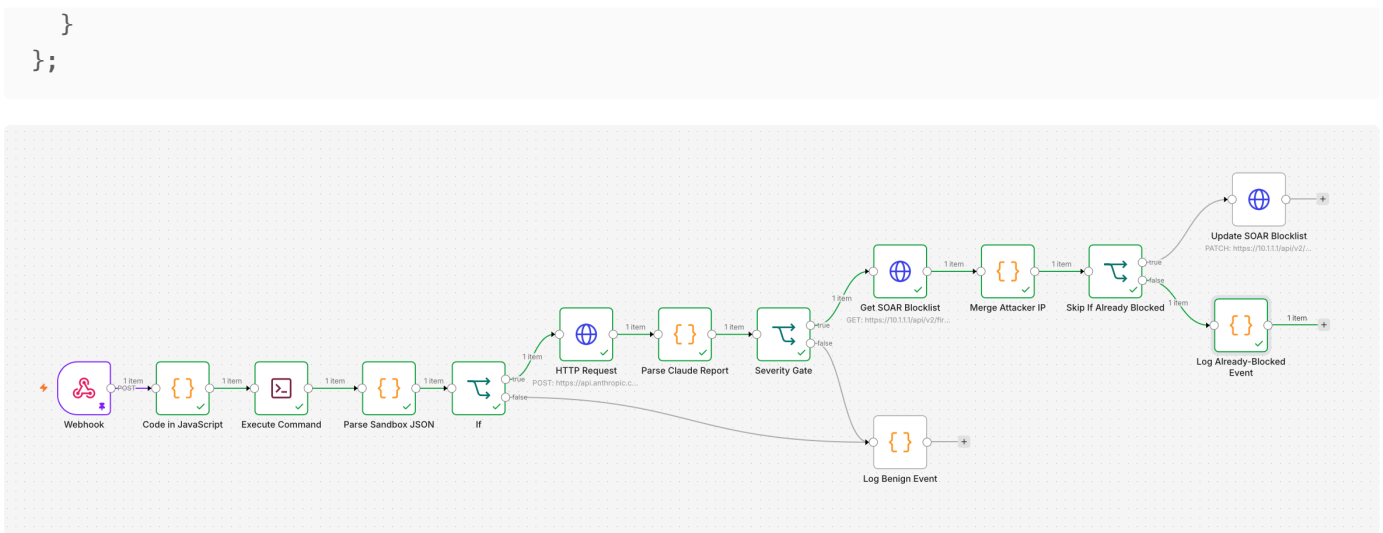
n8n Pipeline Additions

After the sandbox confirms `is_malicious == true`, the workflow now:

1. **Get SOAR Blocklist**: GETs the current alias contents by ID.
2. **Check Early Block Status** (Code node): compares the sandbox's `extracted_ip` against the alias's current address list, before Claude ever runs.
3. **Early Block Gate** (If node): if the IP is already blocked, routes straight to a **Log Already-Blocked Event** node and skips the Claude API call entirely. This follows the same pre-LLM suppression principle from Module 2: don't spend an API call and a human's attention on something already handled.
4. If not already blocked, the alert proceeds to Claude triage as before, then a **Severity Gate** (If node, regex match on `severity` against `^(CRITICAL|HIGH)$`) filters out anything below HIGH from auto-remediation.
5. **Merge Attacker IP** (Code node): merges the attacker's IP into the alias's existing address list, deduplicated via a `Set`.
6. **Update SOAR Blocklist**: PATCHes the alias with `apply: true`.

```
// Merge Attacker IP
const existing = $('Get SOAR Blocklist').first().json.data.address || [];
const attackerIp = $('Parse Claude Report').first().json.attacker_ip;
const updatedList = Array.from(new Set([...existing, attackerIp]));

return {
  json: {
    id: $('Get SOAR Blocklist').first().json.data.id,
    address: updatedList,
    attacker_ip: attackerIp,
    already_blocked: existing.includes(attackerIp)
  }
}
```



Troubleshooting & Key Takeaways

Ordering matters for cost control. My first pass called Claude before checking whether the IP was already blocked, which meant every repeat alert against an already-handled attacker still burned an API call. I reordered the graph so the alias lookup and already-blocked check happen immediately after the sandbox verdict, before Claude is ever invoked. This meant re-pointing `Merge Attacker IP`'s references: it originally read `$json.data.address` assuming it ran right after the GET node, and once the GET moved earlier in the graph, I had to reference `$('Get SOAR Blocklist').first().json` explicitly instead of relying on the immediate upstream `$json`.

Regex vs. string match is a silent trap in n8n's If node. My severity gate initially used operation "is equal to" against a regex pattern string, which does a literal comparison and will never match, silently routing everything to the false branch with no visible error. Switching the operation to "matches regex" fixed it immediately, but it's the kind of bug that looks like a logic error rather than a config error, and cost real debugging time.

Verified with a real trigger, not just pinned data. Testing individual nodes with manually pinned data confirmed the logic worked, but only re-running the actual Kali attack script end-to-end (Splunk detects it live, fires the real production webhook URL, and the pipeline runs unattended) proved the automation works outside of the n8n editor. This surfaced a separate bug: Splunk's alert action was still pointed at n8n's test webhook URL, which only listens while the editor is open. Switching it to the production URL was what made unattended triggering actually work.

Module 5: Human-in-the-Loop Approval via Discord

Why a Human Gate

Auto-blocking on every high-confidence detection is a real production risk. A false positive can lock out legitimate traffic with no one in the loop to catch it. I added a Discord-based approval gate between AI triage and the actual firewall write, so a human reviews Claude's summary and decides whether the block goes through, rather than the pipeline acting fully autonomously.

Discord Bot Setup

I created a dedicated Discord server (SOC–Automation–Lab) rather than reusing an existing one, to keep the bot's permission scope minimal and make the demo self-contained. In the Discord Developer Portal, I registered an application, generated a bot token, and granted it four permissions only: **Send Messages, Embed Links, Read Message History, Add Reactions**. No server management, no member access. I generated an OAuth2 invite URL with scopes `bot` and `applications.commands` and added the bot to the server.

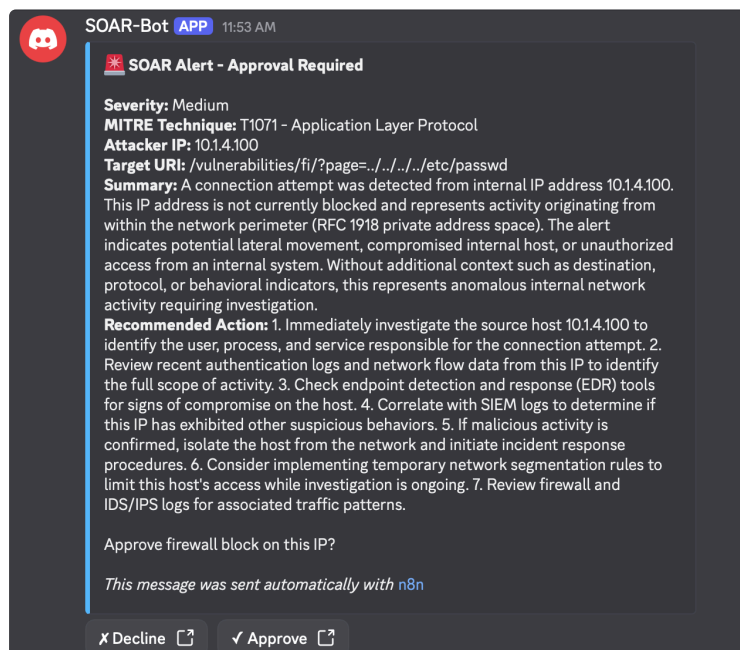
Wiring the Approval Node

n8n's Discord node has a built-in **"Send message and wait for response"** operation with a **Response Type: Approval** mode, which handles the pause-and-resume mechanics natively. No separate webhook workflow needed. I placed it right after **Merge Attacker IP**, so the alert shown to the human already has the resolved attacker IP and Claude's full triage output:

```
SOAR Alert – Approval Required
Severity: {{ $('Parse Claude Report').item.json.severity }}
MITRE Technique: {{ $('Parse Claude Report').item.json.mitre_technique }}
Attacker IP: {{ $json.attacker_ip }}
Target URI: {{ $('Parse Claude Report').item.json.target_uri }}
Summary: {{ $('Parse Claude Report').item.json.incident_summary }}
Recommended Action: {{ $('Parse Claude Report').item.json.remediation_action }}

Approve firewall block on this IP?
```

This posts to the `#soar-alerts` channel with Approve/Decline buttons. Clicking either resumes the paused n8n execution with `data.approved: true/false` in the node's output. It's a single output, not two branches, so I added an **Approval Decision Gate** (If node) afterward to actually route on that value: `true` continues into the existing Skip If Already Blocked > Update SOAR Blocklist chain, `false` routes to a new **Log Denied Block** node.



Troubleshooting & Key Takeaways

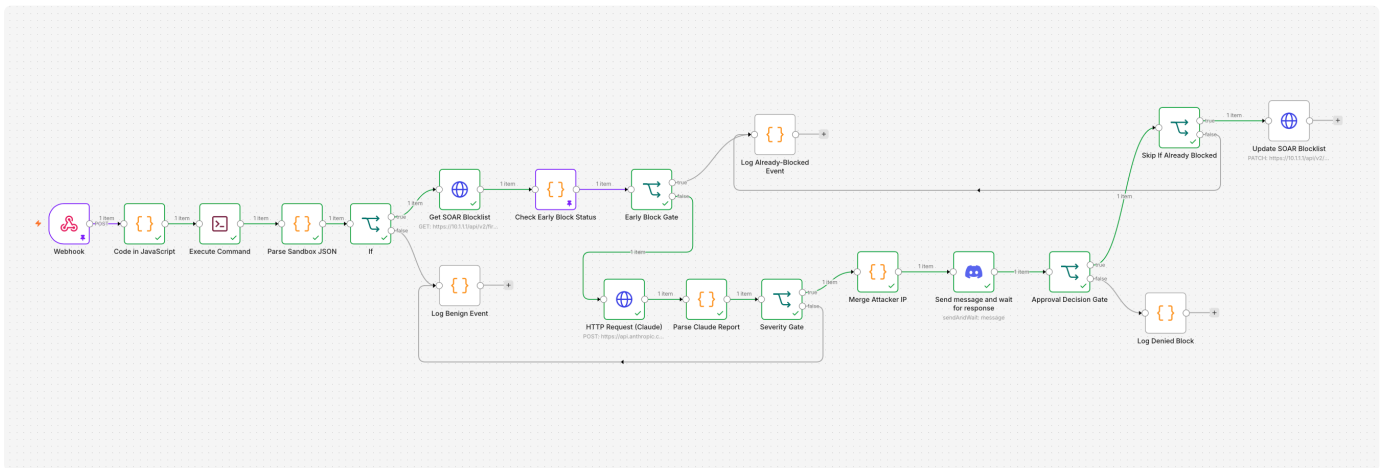
Recreating the n8n container broke more than expected. After running into an expired Splunk API token mid-session, I ended up rebuilding the n8n container to add the environment variables needed for the Discord callback

to work (`WEBHOOK_URL` , `N8N_SECURE_COOKIE=false`), and hit a cascade of separate issues as a result.

I initially ran a bare `docker run` instead of the project's actual `docker-compose.yml` , which meant I lost the Docker socket mount and the `/app` bind mount that Module 2's sandboxed Execute Command node depends on. This resulted in `Unrecognized node type: n8n-nodes-base.executeCommand` and Module 'fs' is disallowed errors once the container came back up on a newer n8n version, which locks these down by default (`NODES_EXCLUDE` , `NODE_FUNCTION_ALLOW_BUILTIN`). Switching to `docker-compose up -d` , which already had these variables correctly set in the project's `.env` , fixed all of it in one step.

I also discovered two separate named Docker volumes for the same project: `n8n_data` from my original Module 1 setup, and `project4-soar-pipeline_n8n_data` auto-created later by Compose's project-prefixed naming. My workflows and credentials were actually living in the Compose-managed volume, not the one I assumed. Worth remembering that switching between `docker run` and `docker compose` for the same project silently creates divergent state if you're not careful about volume names.

Discord's approval node doesn't branch on its own. It was easy to assume Approve/Decline buttons meant two separate output paths on the node itself, the way an If node works. It doesn't. Both outcomes flow through the same single output with a boolean field, and you need an explicit If node afterward to route the True/False cases.



Module 6: Closed-Loop Audit Logging to Splunk

Why Write Back to the SIEM

After building Modules 1 through 5, the pipeline had a real gap: every decision (benign suppression, already-blocked skip, human denial, firewall block applied) dead-ended in n8n's execution history. That's fine for debugging during the build phase, but it's not an audit trail. Without a durable, searchable record of SOAR decisions alongside the original detection events, there's no way to answer basic SOC questions like "what happened to that alert from last Tuesday" or "show me every block action this pipeline took this week."

I wanted the SIEM to be the single source of truth for both detections and responses, not just detections. Splunk already held the raw alert data. Sending outcomes back to the same platform closes the loop.

Splunk HTTP Event Collector Setup

Splunk's HTTP Event Collector (HEC) is the standard REST interface for writing events directly into a Splunk index. I created a dedicated index (`soar_audit`) so SOAR outcomes are stored separately from raw log sources,

making them trivial to query independently.

In the Splunk web UI, I created a new HEC token:

- **Name:** `n8n-soar-writeback`
- **Source type:** `_json` (Splunk auto-extracts JSON keys as searchable fields)
- **Default Index:** `soar_audit`
- **Allowed Indexes:** `soar_audit` only (locked to prevent accidental writes elsewhere)

I confirmed HEC was globally enabled (Settings > Data Inputs > HTTP Event Collector > Global Settings > All Tokens: Enabled).

Design Choice: One Shared Node, Not Five

I had five distinct pipeline outcomes to write back: benign suppression, already-blocked skip, denied via Discord, approved and blocked, plus the false severity gate path. Each already produced its own JSON object from a dedicated Code node. Rather than add five separate HTTP Request nodes, I routed all five outputs into a single shared "Send to Splunk" node.

The tradeoff is slightly more complex wiring (five lines converging on one input), but the benefit is real: one place to debug when something changes at the HEC layer, and a consistent payload shape (same index, same sourcetype, same auth header) enforced everywhere by design rather than by convention.

I also added a small "Log Approved Block" Code node right after `Update SOAR Blocklist`. Until Module 6, the block success path produced no logged record at all. It now emits a `BLOCK_APPLIED` event with the attacker IP, severity, and timestamp, joining the other four outcomes in the write-back flow.

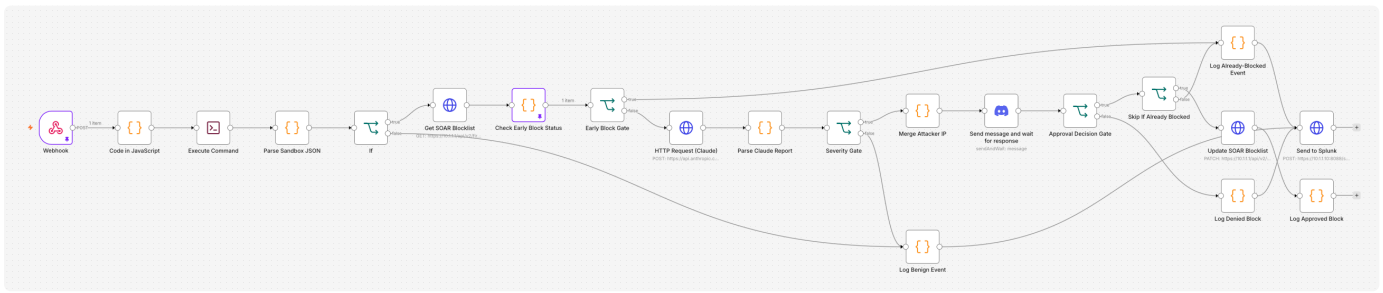
Wiring the HEC POST

The HTTP Request node configuration:

- Method: POST
- URL: `https://10.1.1.10:8088/services/collector/event`
- Send Headers:
 - `Authorization = Splunk <token>`
 - `Content-Type = application/json`
- Options: Ignore SSL Issues (Insecure) enabled

The body wraps each log node's existing JSON output as Splunk's `event` field:

```
{
  "index": "soar_audit",
  "sourcetype": "_json",
  "event": {{ JSON.stringify($json) }}
}
```



I verified the write landed by running the attack script end-to-end (fresh unblocked IP, approved on Discord, firewall PATCH confirmed) and then searching `index=soar_audit status=BLOCK_APPLIED` in Splunk. The event appeared with `attacker_ip`, `severity`, and `timestamp` auto-extracted as searchable fields.

New Search SPL Convert to SPL2 Save As Create Table View Close

`index=soar_audit status=BLOCK_APPLIED` Time range: Last 24 hours Q +

✓ 1 event (9/13/26 10:00:00.000 AM to 9/14/26 10:55:29.000 AM) No Event Sampling Job || → ⌵ ⌴ ⌵ ⌴ Smart Mode

Events (1) Patterns Statistics Visualization

Timeline format Zoom Out + Zoom to Selection x Deselect 1 hour per column

Format Show: 20 Per Page View: List

< Hide Fields	All Fields	i	Time	Event
SELECTED FIELDS <a>host 1 <a>source 1 <a>sourcetype 1 INTERESTING FIELDS <a>action_taken 1 <a>attacker_ip 1 <a>index 1		>	9/14/26 10:54:34.000 AM	<pre>{ [-] action_taken: Firewall block applied to SOAR_Blocklist attacker_ip: 10.1.4.100 severity: Medium status: BLOCK_APPLIED timestamp: 2026-09-14T08:53:36.023Z }</pre> <a>Show as raw text host = 10.1110:8088 source = http:n8n-soar-writeback sourcetype = _json

Troubleshooting & Key Takeaways

HEC auth is `SpLunk <token>`, not `Bearer <token>` and not just the raw token. My first POST attempt returned 401 until I prefixed the token value with the literal word `SpLunk` followed by a space. The error message is generic (just "401 Unauthorized") and doesn't hint at the format issue at all. This is a common pitfall because n8n's Header Auth manager defaults to `Bearer` prefixing, and most REST APIs follow that convention. Splunk HEC doesn't.

Discord's approval node silently replaces `$json`. The "Send message and wait for response" node wipes the data context with its own approval result (`data.approved: true/false`), so any node downstream of it (like `Update SOAR Blocklist`) can't use `$json` or `{{ $json.field }}` expressions to reference earlier pipeline data. I hit this when `Update SOAR Blocklist` suddenly produced invalid JSON because `{{ $json.id }}` resolved to nothing. The fix is to reference upstream nodes explicitly: `$( 'Merge Attacker IP' ).first().json.id` instead of `$json.id`. This tripped me in Module 4 as well with the `Get SOAR Blocklist` reference. It's one of those n8n patterns that works until a node in the middle decides to be the new owner of `$json`, and then it breaks silently.

Overall Key Takeaways

Suppress early, spend late. The most impactful design decision in this project wasn't any single integration. It was the pre-LLM suppression pattern: the Python sandbox scores risk locally and filters benign events before they reach Claude, and the already-blocked check filters repeat offenders before they reach Claude or a human. Each filtering layer saves money and attention at the next, more expensive stage. This isn't just about API costs. In a real SOC, it's about analyst fatigue: every false alarm that reaches a human burns trust in the automation.

Verify with real triggers, not just pinned data. Multiple bugs were invisible in manual node-by-node testing and only surfaced when the full pipeline ran unattended: the Splunk webhook action pointed at n8n's test URL (silently failing once the editor was closed), the Discord approval node overwriting `$json` and breaking downstream references, the severity gate's regex silently routing everything to the false branch because the operation was set to "is equal to" instead of "matches regex." Pinned data is good for building. Real triggers are the only thing that proves the automation actually works.

Read-modify-write has a race condition, and that's worth documenting honestly. The pfSense alias update is a GET-then-PATCH pattern. If two alerts for different attacker IPs fired close enough together, the second PATCH would overwrite the first's addition, silently dropping a block action. I didn't solve this in the lab (a production fix would move the list merge server-side or serialize writes through a queue), but I'm documenting it as a known limitation because it's worse to pretend the system has no edge cases than to show I understand the failure mode and can reason about it. That transparency is something I'd want from any engineer I worked with, and it's how I'd want my own portfolio read.